Aurora Computer Studies

#1 in Quality for ICT Education in Sri Lanka

Data Structures & Algorithms

Topic 6 – Analysis of Algorithms

Bachelor of Information Technology (BIT)

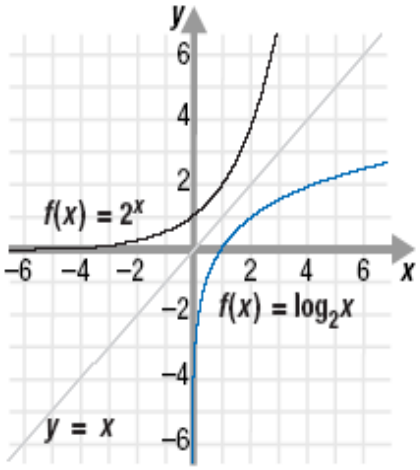
Level 2 (Semester 3)

1 Analysis of Algorithms & Big O Notation

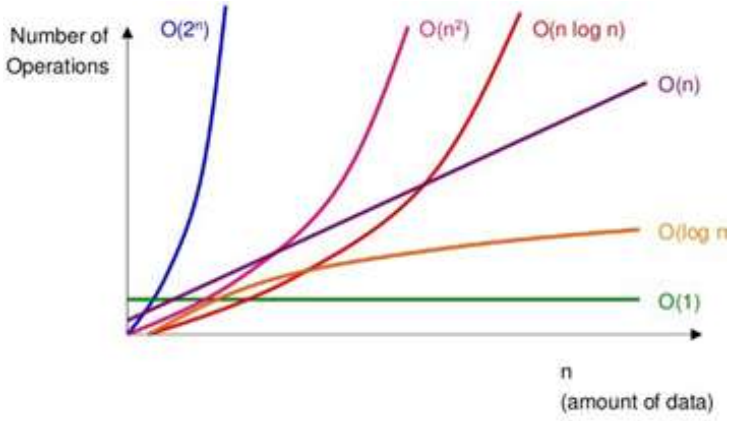
1.1 Revision on Mathematics

1.1.1 2^x Vs $\log_2 x$ Graph

- 2^x is inverse function of $\log_2 x$
- For any positive number c,
  $\log_2 x < x < 2^x$



1.1.2 Comparing Graph of Widely Used Mathematical Functions



1.2 Major Concerns Related to the Efficiency of Algorithms

- ***Time/Speed***
  - Since time is a precious resource—computer solutions should run as fast as possible
  - This is our primary analysis factor
- ***Space (memory) Usage***
  - Lesser the memory consumption, the better

Note: Time-space tradeoffs

1.3 Factors Effecting Performance

- Algorithm
    - Process logic
- Input data size
- Hardware configuration
- Software environment (Operating system, programming language, compiler, interpreter, etc)

1.4 Techniques of Finding the Efficiency/speed of an Algorithm

- Experimental/Statistical studies
    - Experimented oriented
- Analysis of algorithms
    - Mathematical oriented

1.5 Drawbacks of Experimental Studies

- Comparing the experimental running times of two algorithms are unrealistic unless the experiments were performed in exactly the same hardware and software environments
- Need to fully implement and execute an algorithm in order to study its running time experimentally

1.6 Analysis of Algorithms

If we wish to analyze a particular algorithm without performing experiments on its running time, we can perform an analysis directly on the high-level pseudo-code instead.

We define a set of primitive operations in high-level algorithms;

- Assigning a value to a variable
- Calling a method
- Performing an arithmetic operation (for example, adding two numbers)
- Comparing two numbers
- Indexing into an array
- Following an object reference
- Returning from a method

Instead of trying to determine the specific execution time of each primitive operation, we will simply count how many primitive operations are executed, and use this number *t* as a measure of the running-time of the algorithm.

1.6.1 Worst Case Efficiency

- Worst case refers to the running time of an algorithm as the maximum taken over all inputs of a same size

1.6.2 Best Case Efficiency

- The minimum number of steps (time) that an algorithm can take any collection of data values.

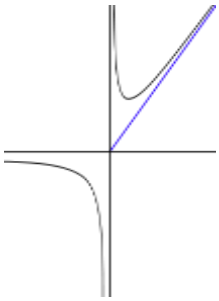
1.6.3 Average Case Efficiency

- Average case refers to the running time of an algorithm as an average taken over all inputs of a same size
- Must assume a distribution of the input
- we normally assume uniform distribution (all keys are equally probable)

1.7 Efficiency of Algorithms - Big O Notation

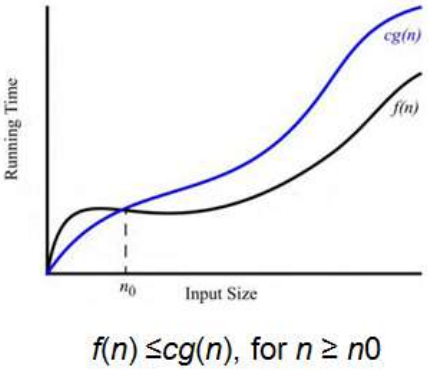
1.7.1 Asymptotic Analysis

- In analytic geometry, an asymptote of a curve is a line such that the distance between the curve and the line approaches zero as they tend to infinity
- In computer science and applied mathematics, particularly the analysis of algorithms, asymptotic analysis is a method of describing limiting behavior. Examples include the performance of algorithms when applied to very large input data, or the behavior of physical systems when they are very large.



1.7.2 Big-O Notation

- Big O notation is used in Computer Science to describe the performance or complexity of an algorithm
- Big O specifically describes the worst-case scenario particularly for large data, and can be used to describe the execution time required or the space used (e.g. in memory or on disk) by an algorithm
- Let f(n) and g(n) be functions mapping nonnegative integers to real numbers. We say that f(n) is O(g(n)) if there is a real constant c > 0 and an integer constant n_0 ≥ 1 such that;
  f(n) ≤ cg(n), for n ≥ n_0
  - This is sometimes pronounced as "f(n) is big-Oh of g(n)"



$f(n) \leq cg(n)$, for $n \geq n0$

- Alternatively, we can also say "f(n) is order of g(n)"

- The big-Oh notation gives an upper bound on the growth rate of a function
- The statement “f(n) is O(g(n))” means that the growth rate of f(n) is no more than the growth rate of g(n)
- We can use the big-Oh notation to rank functions according to their growth rate

1.7.3 Big O Rules

1. If is f(n) a polynomial ($f(n) = a_0 + a_1 n + a_2 n^2 + \ldots + a_d n^d$) of degree d, then f(n) is $O(n^d)$, i.e.,
   - Drop lower-order terms
   - Drop constant factors
2. Use the smallest possible class of functions
   - Say “2n is O(n)”
3. Use the simplest expression of the class
   - Say “3n + 5 is O(n)”
4. Examples
   - We say that $n^4 + 100n^2 + 10n + 50$ is of the order of n^4 or $O(n^4)$
   - We say that $10n^3 + 2n^2$ is $O(n^3)$
   - We say that $n^3 - n^2$ is $O(n^3)$
   - We say that 10 is O(1),
   - We say that 1273 is O(1)

1.7.4 Other Asymptotic Notations for Complexity of Algorithms

- Big - Omega
- Big - Theta
- Little Oh
- Little – Omega

Big Omega

- It describes <u>the best</u> that can happen for a given data size
- The omega is on the opposite side. When you say an algorithm’s, complexity is Omega(f(n)), it means that its complexity is <u>equal to or worse than</u> f(n)

Big - Theta

- This is basically saying that the function, f(n) is bounded both from the top and bottom by the same function, g(n)

.

For class details visit auroracs.lk/bit or call 071 984 2030

auroracs.lk/bit

youtube.com/auroracslk